

第 3 週：組合邏輯與循序邏輯

本週一句話： `always` 有兩種用法，用錯符號會做出完全不同的電路。

這一週是整個課程**最重要的一週**。前面學的是語法，這裡開始學「怎麼寫才對」。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week03_comb_seq
```

每個範例都是這三條（外加一條檢查），把 `04_blocking_vs_non` 換成你要跑的名字：

```
iverilog -o sim.out 04_blocking_vs_non.v 04_blocking_vs_non_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_blocking_vs_non.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_case_mux`
- `02_latch_trap`
- `03_dff`
- `04_blocking_vs_non`
- `05_shift_reg`
- `06_comb_blocking` ★ 組合邏輯用錯 `<=` 的後果

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] 分得出 `always @(*)` 和 `always @(posedge clk)`
- [] ★★ 說得出 `=` 和 `<=` 做出來的電路差在哪
- [] 知道什麼是 `latch`，以及為什麼要避免
- [] 會寫 D 型正反器的同步 / 非同步 reset
- [] 會用 `case` 寫多工器和解碼器

一、兩種 `always`

	組合邏輯	循序邏輯
寫法	<code>always @(*)</code>	<code>always @(posedge clk)</code>
何時執行	輸入一變就重算	只在時脈上升緣
賦值符號	<code>=</code>	<code><=</code>
有沒有記憶	✗ 沒有	✓ 有
綜合成	邏輯閘	正反器 + 邏輯閘

[!IMPORTANT] 口訣 (背起來，一輩子受用)

```
always @(posedge clk) 裡 → 一律用 <=
always @(*)            裡 → 一律用 =
```

業界規範就是這樣，不要混用。

為什麼 `output` 有時要寫 `reg` ？

```
output reg [3:0] y    // 因為 y 在 always 裡被賦值
output wire [3:0] y   // 因為 y 用 assign 賦值
```

規則：在 `always` 裡被賦值 → 宣告成 `reg`；用 `assign` → `wire`。

⚠ 注意：`reg` 不代表它會變成正反器。`always @(*)` 裡的 `reg` 綜合出來是純組合邏輯，一顆正反器都沒有。這個關鍵字取名取得很爛，是 Verilog 有名的坑。

二、★★ 本週核心：`=` vs `<=`

同樣三行程式碼：

<code>q1 <= d;</code>	// 非阻塞	<code>q1 = d;</code>	// 阻塞
<code>q2 <= q1;</code>		<code>q2 = q1;</code>	
<code>q3 <= q2;</code>		<code>q3 = q2;</code>	

	非阻塞 <code><=</code>	阻塞 <code>=</code>
執行方式	先把右邊全部讀好，再一起寫左邊	一行做完才做下一行（像寫軟體）
q2 讀到的 q1	上一拍的舊值	這一拍剛寫入的新值
做出來的電路	3 顆正反器串起來	塌成 1 顆

實測結果（範例 04 的真實輸出）

		非阻塞 <code><=</code>			阻塞 <code>=</code>		
		q1	q2	q3	q1	q2	q3
第 1 拍	d=1		1	0		1	1
第 2 拍	d=0		0	1		0	0
第 3 拍	d=0		0	0		0	0
第 4 拍	d=0		0	0		0	0

看得出來嗎？

- 非阻塞：那個 1 一格一格往後走 `q1 → q2 → q3`，走了 3 拍 → 真的是 3 顆正反器
- 阻塞：第 1 拍三個同時變 1，第 2 拍就全沒了 → 只有 1 顆正反器

只換一個符號，電路數量差 3 倍。這就是為什麼要有口訣。

三、latch 陷阱

```
always @(*) begin
    if (en)
        y = d;
    // ← 沒有 else !
end
```

`en=0` 的時候沒說 `y` 要等於什麼，Verilog 只好讓它「保持上一次的值」——於是憑空生出一個會記憶的元件，叫做**門鎖器 (latch)**。

為什麼是大忌？

- 你以為在寫組合邏輯，卻做出了有記憶的東西 → 行為跟你想的不一樣
- latch 沒有時脈，FPGA 的時序分析工具**無法分析它** → 時序無法保證
- 綜合工具通常只給一行小小的 warning，很容易被忽略

兩種修法：

```
// 修法 1：補 else
always @(*) begin
    if (en) y = d;
    else   y = 4'b0;
end

// 修法 2（推薦）：最上面先給預設值
always @(*) begin
    y = 4'b0;           // 預設
    if (en) y = d;       // 有條件才覆蓋
end
```

`case` 也一樣 —— 一定要寫 `default`。

四、本週五個範例

```
cd week03_comb_seq
```

01_case_mux — case 寫多工器與解碼器

```
iverilog -o sim.out 01_case_mux.v 01_case_mux_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_case_mux.gtkw
```

看什麼： `always @(*)` 的組合邏輯行為； `decoded` 永遠只有一位是 1 (one-hot 編碼，第 6 週會用到) 。

02_latch_trap — latch 陷阱 ★

```
iverilog -o sim.out 02_latch_trap.v 02_latch_trap_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_latch_trap.gtkw
```

看什麼：波形上 `bad` 和 `good` 的對比。 `en=0` 時 `good` 乖乖變 0，`bad` 卡住不動——那就是 latch 在記憶。

實測：

en	d		bad	good	
1	A		a	a	兩個都跟著 d
0	A		a	0	★ bad 卡住不放！
0	5		a	0	★ d 變了 bad 還是舊值

03_dff — D 型正反器與兩種 reset

```
iverilog -o sim.out 03_dff.v 03_dff_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_dff.gtkw
```

看什麼：★ 波形上 `t=76` 附近那個只有 4ns 的 `rst` 短脈衝。

- `q_async` 立刻被清成 0 (不等時脈)
- `q_sync` 完全沒反應 (脈衝結束前沒遇到上升緣)

這就是你之前問的「為什麼 reset 拉高了 count 還沒歸零」的完整答案。

📁 04_blocking_vs_non — ★★ 本週最重要

```
iverilog -o sim.out 04_blocking_vs_non.v 04_blocking_vs_non_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_blocking_vs_non.gtkw
```

看什麼：波形上把 `q1_nb` `q2_nb` `q3_nb` 和 `q1_b` `q2_b` `q3_b` 排在一起。非阻塞那三條是**階梯狀**依序點亮，阻塞那三條**同時亮**又**同時滅**。

看懂這張波形，`=` 和 `<=` 就不用背了。

這個範例示範的是「時脈區塊用錯 `=`」。另一半（組合區塊用錯 `<=`）在下面的 `06_comb_blocking`。

📁 05_shift_reg — 移位暫存器

```
iverilog -o sim.out 05_shift_reg.v 05_shift_reg_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 05_shift_reg.gtkw
```

看什麼：`q` 這條 8 位元的線整體往左或往右「滑」。把本週和第 2 週學的 `case` + `<=` + 位元串接全用上了。

實測：

載入 8'hC3	11000011	
左移 sin=1	10000111	← 整條往左滑一格，右邊補 1
左移 sin=0	00001110	
保持	00001110	← 完全不動
右移 sin=1	10000111	← 往右滑，左邊補 1

📁 06_comb_blocking — 組合邏輯用錯 `<=` ★

```
iverilog -o sim.out 06_comb_blocking.v 06_comb_blocking_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 06_comb_blocking.gtkw
```

04 的另一半。電路是 `y = (a & b) | c`，兩種寫法並排。

實測：

```
a=1 b=1 c=0 | 剛設定完 : y_ok=1   y_bad=0       <- y_bad 錯的
              | 等 5ns 後: y_ok=1   y_bad=1       <- 才追上

a=0 b=0 c=0 | 剛設定完 : y_ok=0   y_bad=1       <- 又錯
              | 等 5ns 後: y_ok=0   y_bad=0       <- 才追上
```

`y_bad` 每次都先閃一個錯的值才追上——這就是毛刺。

原因：`<=` 會「先把右邊全部讀好」，所以第二句讀中間值 `tmp2` 的時候，第一句還沒寫進去，讀到的是上一次的。

兩邊用錯的後果剛好相反：

- 時脈區塊用 `=` → 正反器塌掉（範例 04）
- 組合區塊用 `<=` → 產生毛刺（這個範例）

這就是為什麼口訣不能破例。

五、練習題

練習 1

把 `02_latch_trap.v` 裡的 `bad` 修好（兩種修法都試一次），重跑確認 `bad` 和 `good` 波形完全一樣。

練習 2

寫一個 4 位元環形計數器（ring counter）：reset 後是 `0001`，每個時脈變成 `0010` → `0100` → `1000` → `0001` 循環。

提示：一行就好，用位元串接。

練習 3

寫一個 8 位元的可預載上下數計數器：

- `load=1` → 載入 `data_in`
- `up=1` → 加 1 ; `up=0` → 減 1
- 優先順序：`rst` > `load` > 計數

練習 4 (思考)

下面這段會做出幾顆正反器？

```
always @(posedge clk) begin
    a = b;
    b = c;
    c = a;
end
```

► 看答案

六、本週檢核

- [] 五個範例都跑過，波形都看過
- [] 04 的波形能指出「哪三條是階梯狀、哪三條是同時變」
- [] 02 的波形能指出 `bad` 卡住的那幾段
- [] 說得出為什麼 latch 是大忌 (兩個理由)
- [] 練習 2、3 寫出來而且測試通過
- [] 練習 4 答對

[!TIP] 這週卡住很正常 —— `= / <=` 是所有人的關卡。卡住就回去把 04 的波形多看幾次，比讀十遍文字有效。

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔 (`sim.bat`)，不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。